College Admission Management System

Project report | Relational Database Management Systems

Assignment: Case 1 - Project Report + Live Demo (5 + 15 marks)

Student name: _____

Roll number: _____

Institution / department: _____

Instructor: _____

Submission date: _____

Technology: React UI + Express REST API + MySQL / InnoDB

This report describes the design and implementation. Fill in the cover details and attach screenshots from the completed MySQL demonstration before submission.

1. Problem, objectives and architecture

The College Admission Management System is an administrative website for student registration, course applications, admission review, hostel allocation and bus passes. Admission cycles, announcements, review history and a manual payment ledger extend the workflow.

Independent spreadsheets and paper forms can duplicate records, hide seat availability and lose the history of decisions. A relational database centralizes records and enforces their relationships.

Assignment objectives

• Prepare a report covering the problem, design, implementation and verification.

• Develop an interface connected to MySQL for persistent administrative operations.

• Demonstrate UI changes in MySQL through SELECT queries, refresh and reporting.

• Validate input, prevent duplicate active allocations and use atomic transactions.

Request and data flow

Layer	Responsibility
React / Vite	Forms, tables, charts, routes and error messages
Express API	JWT authentication, request validation and business rules
mysql2 pool	Parameterized SQL and transaction connections
MySQL / InnoDB	Persistent tables, keys, constraints, views and routines

Browser form -> REST request -> validation -> SQL transaction -> JSON response -> refreshed interface.

Environment: Node.js 20.19+ or 22.12+, npm, MySQL 8.0.16+, modern browser. Local ports: UI 5173; API 5001; MySQL 3307. Credentials live in server/.env. A MySQL failure is displayed as disconnected; there is no SQLite substitution.

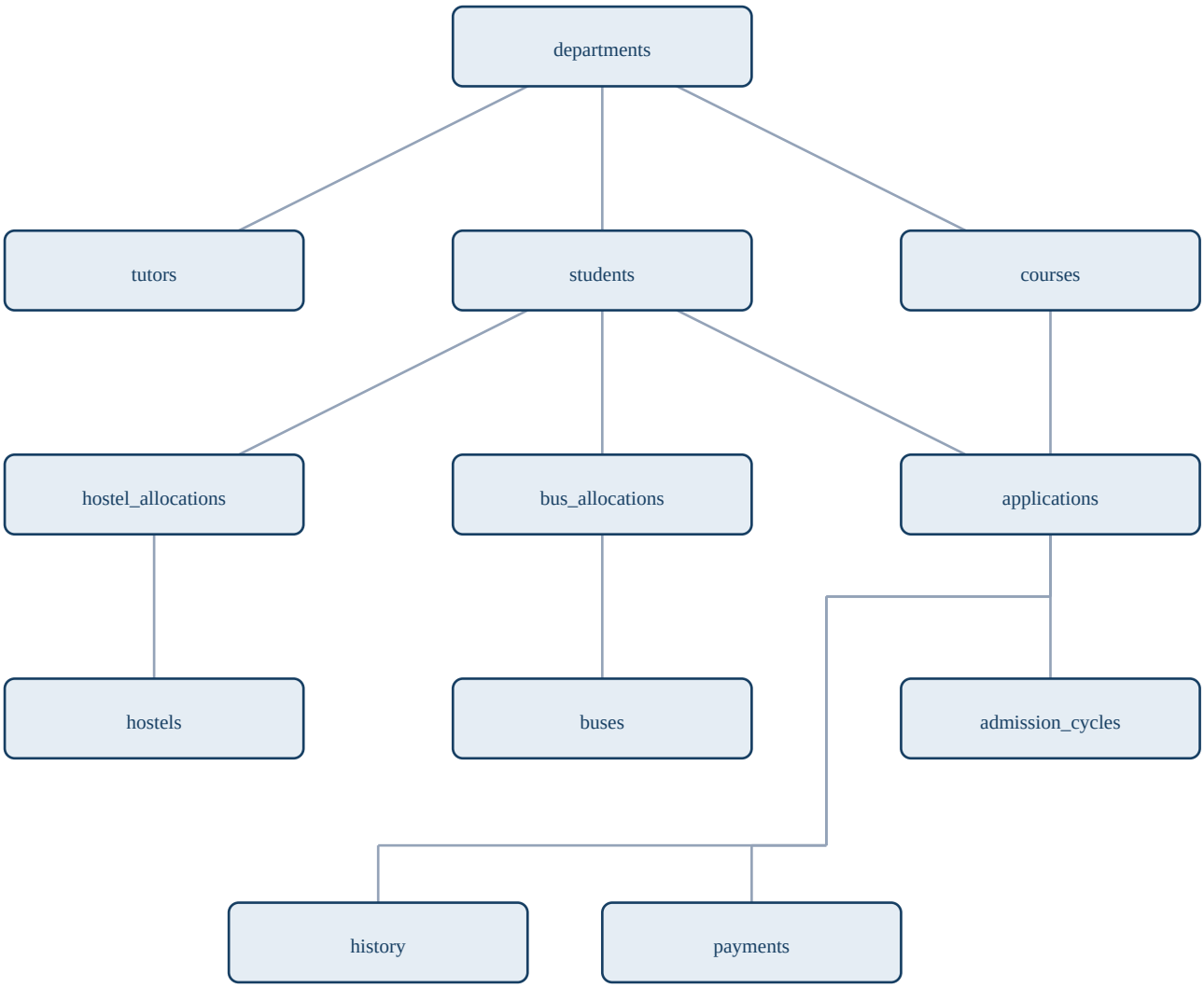
2. Relational schema and ER model

Relation	PK	Relationship / purpose
users	user_id	Optional applicant accounts
admins	admin_id	Administrator accounts
departments	department_id	Academic department catalog
courses	course_id	Department FK; program and capacity
tutors	tutor_id	Department FK; faculty records
students	student_id	User/department FKs; academic dossier
admission_cycles	cycle_id	Session dates and status
applications	application_id	Student, course, cycle FKs
application_status_history	history_id	Application FK; decision timeline
hostels	hostel_id	Facility and bed capacity
hostel_allocations	allocation_id	Student and hostel FKs
buses	bus_id	Routes and passenger capacity
bus_allocations	allocation_id	Student and bus FKs
announcements	announcement_id	Administrative notices
payments	payment_id	Application FK; manual fee ledger
audit_logs	log_id	Database state-change records

A department has many courses, tutors and assigned students. Applications bridge students and courses, with an optional admission cycle. Each application has many history/payment rows. Allocation rows preserve student-facility assignment history.

Each student may hold one approved admission, one active hostel bed and one active bus pass. Historical inactive allocations remain stored.

Entity-relationship diagram



Each line represents a foreign-key relationship: one parent may have many child rows. Applications and allocation tables connect students to courses and facilities. The history box represents application_status_history.

Users link optionally to students. Administrator credentials, announcements and audit logs support the portal independently of the main admission relationships.

3. Normalization and integrity

1NF: Scalar fields hold one value. Applications and allocations are individual rows rather than repeating lists.

2NF: Entity attributes depend on the entity key; dates and statuses of relationships belong to their bridge rows.

3NF: Course/department/facility details live in separate relations and are referenced through foreign keys. Counts and available seats are derived.

Mechanism	Rule
Primary / foreign keys	Identify records; enforce valid parent-child references
Unique constraints	Emails, application references and transaction references
Generated conditional keys	One approved admission / active allocation per student
Generated cycle key	Duplicate application protection even when cycle_id is NULL
CHECK + API validation	Mark/age/capacity boundaries, valid dates, positive payment amounts
Parameterized SQL	Form data is bound as values rather than SQL text

Transactions

Creation writes the application and initial history together. Review updates status, review fields, history and enrollment in one transaction. Approval checks course capacity and existing approved admissions. Revocation/deletion synchronizes the student department.

Allocation creation and reactivation lock the student and resource, check duplicates and occupancy, and then insert/update. Capacity edits use the same resource locks. READ COMMITTED makes occupancy reads observe commits after lock waits. Errors roll back; connections are released.

Authentication: bcrypt hashes passwords and JWT tokens authenticate API requests. Only the admins table permits entry to the administrator portal.

4. SQL and implementation

Views: view_department_stats, view_hostel_occupancy, view_bus_occupancy calculate department totals and available facilities.

Routines: merit and availability functions; approval, hostel allocation and department report procedures. The routines/triggers file is imported separately. The API viva page reads information_schema so it lists installed objects accurately.

```
SELECT s.name, c.course_name, d.department_name, a.status
FROM applications a
JOIN students s ON s.student_id = a.student_id
JOIN courses c ON c.course_id = a.course_id
JOIN departments d ON d.department_id = c.department_id
WHERE a.status = 'APPROVED';

SELECT * FROM view_hostel_occupancy;

SELECT department_id, COUNT(*) AS student_count
FROM students
GROUP BY department_id;
```

The SQL demonstration runner uses read-only transactions, rejects modification/file export statements and refuses multiple statements. CSV reports escape text that could otherwise become a spreadsheet formula.

Payments are administrative ledger records. The project does not perform real payments or integrate a payment gateway. Applicant self-service is future work.

5. Verification and live demonstration

Automated controller/validation checks passed: 48 tests, 0 failures. The frontend production build passed with a non-fatal large-bundle warning. Some tests use controlled database mocks; they do not establish live MySQL connectivity.

The db:check command verifies real MySQL, all 16 tables and curated SQL demonstrations. On success it saves docs/MYSQL_VERIFICATION.json with version, installed objects and query results. Live verification passed against MySQL 26.7.0 on 127.0.0.1:3307, database college_submission: all 16 tables, 12 SQL demonstrations and 16 workflow checks. Chrome login and the connection diagnostics were also verified.

Live demonstration sequence

1. Sign in; show Settings and the MySQL connection status.

2. Register a fictional student; show the row in Workbench with SELECT.

3. Create and approve an application; show its history and enrollment.

4. Allocate a hostel bed/bus pass; demonstrate duplicate/full rejection.

5. Export a CSV and execute JOIN, aggregate, subquery and view examples.

6. Refresh the UI to show persistence. Explain keys, normalization and transactions.

Conclusion and next steps

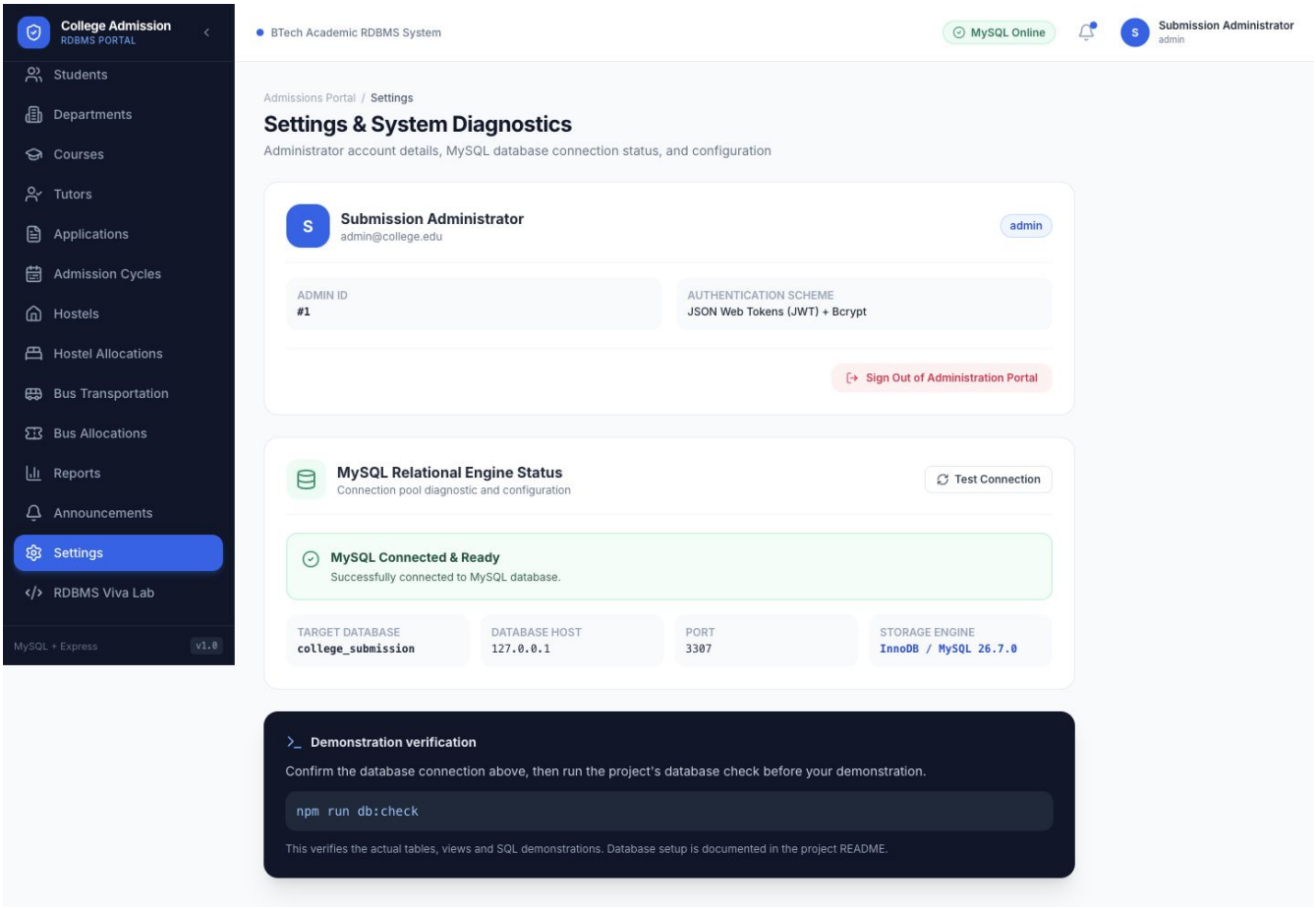
The project combines a functional administrative interface with a relational schema, integrity constraints, transactions, joins and views. Completion of the live MySQL checklist demonstrates the assignment requirements. Fill in the report cover and append demonstration screenshots.

References

https://dev.mysql.com/doc/refman/8.4/en/
https://dev.mysql.com/doc/refman/8.4/en/innodb-locking-reads.html
https://expressjs.com/
https://react.dev/

6. Live UI and MySQL evidence

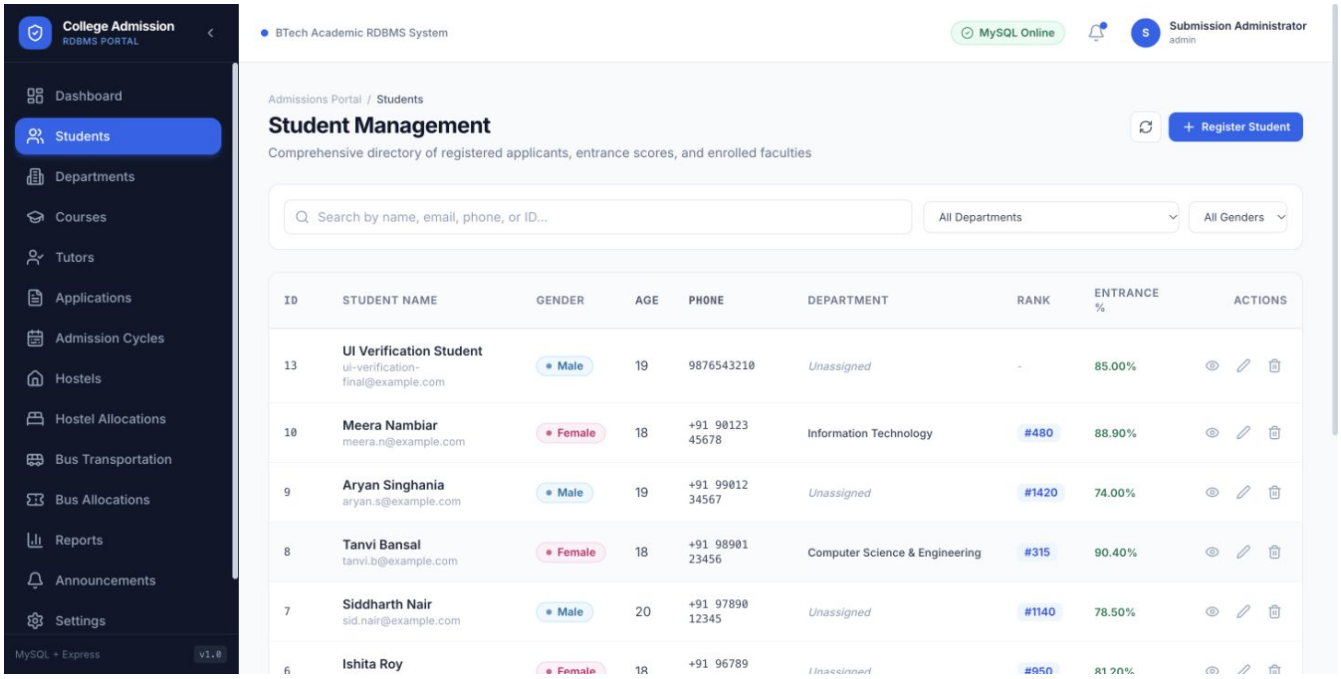
The connection diagnostic shows the actual database, host, port and MySQL version. The student registration screenshot shows a browser-submitted fictional test record; its persistence was confirmed by a direct SELECT query and the fixture was then removed.



Verified connection to college_submission on MySQL 26.7.0, port 3307.

7. Student registration evidence

A fictional student was submitted through the UI and appeared in the list with its MySQL-assigned ID. The saved UI_VERIFICATION.json records the direct database check. The test row was removed afterwards.



Student creation through the React UI, confirmed in MySQL.